TECHNICAL ARCHITECTURE PLAN

AI-Integrated Mobile Application for Remote Skin Analysis

Keras Fine-Tuned Skin Condition Classifier + React Native

Student	Otieno Cyrus Omondi
Registration No.	084935
Programme	MSc Mobile Technology and Innovation
Institution	Strathmore University – Safaricom Academy
Industry Partner	Dr Rashel Kenya
Document Version	v1.0 – Initial Architecture Draft
Date	February 2026

1. Introduction & Rationale for Keras Integration

This document defines the technical architecture for an AI-powered mobile application that enables remote skin analysis and personalized skincare product recommendations for Dr Rashel Kenya. Rather than building a computer vision model from scratch, this project leverages Keras — the high-level deep learning API built on TensorFlow — to fine-tune a pre-existing neural network on labelled skin condition data.

This approach delivers three core advantages:

- Faster time-to-value: Fine-tuning a pre-trained Keras model (e.g., MobileNetV2 or EfficientNetB0) requires significantly less data and compute than training from zero, making it feasible within an academic project timeline.
- Proven accuracy: Pre-trained models have already learned rich visual feature representations from millions of images (ImageNet), reducing the risk of underfitting on a limited skin dataset.
- Mobile-ready deployment: Keras models convert natively to TensorFlow Lite (.tflite), which integrates directly into React Native via the react-native-tflite library for on-device inference.

2. High-Level System Architecture

The system is composed of four interconnected layers: the React Native mobile client, the Keras-powered AI engine, a RESTful backend API, and the existing Dr Rashel web platform. The diagram below describes the data and control flow between these layers.

Layer	Component	Responsibility
Layer 1	React Native App	UI, camera capture, questionnaire, results display, product ordering, loyalty module
Layer 2	On-Device TFLite Model	Real-time skin analysis from camera image (fine-tuned Keras model exported to .tflite)
Layer 3	FastAPI Backend	Model inference fallback, recommendation engine, user profile management, Bitmoji sync, order processing
Layer 4	Keras Training Pipeline	Model fine-tuning on labelled skin data, evaluation, export to TFLite and SavedModel formats
Layer 5	Dr Rashel Web Platform	Inventory, order management, CRM – connected via REST API to the FastAPI backend

3. Keras Skin Condition Classifier

3.1 Model Selection & Rationale

The classifier will be built using transfer learning on EfficientNetB0, a Keras application model that offers an optimal balance between accuracy and model size — critical for mobile deployment. EfficientNetB0 is pre-trained on ImageNet and will be fine-tuned on a curated skin condition dataset.

Criterion	EfficientNetB0	MobileNetV2	ResNet50
Model size	~20MB	~14MB	~98MB
Top-1 ImageNet Acc.	77.1%	71.8%	74.9%
Mobile suitability	High	Very High	Low
Fine-tuning ease	High	High	Medium
Verdict	✓ Selected	Fallback option	Not suitable

3.2 Dataset Strategy

The model will be fine-tuned using a combination of:

- ISIC Archive (International Skin Imaging Collaboration) – publicly available labelled dermoscopy images covering acne, hyperpigmentation, and other conditions.
- In-store Bitmoji analyzer outputs – exported skin profiles (with customer consent) from Dr Rashel's existing analyzer, providing local Kenyan skin tone representation.
- Questionnaire-augmented labels – skin type labels (oily, dry, combination, normal, sensitive) derived from user questionnaires, used as supplementary ground truth.

Target output classes for the classifier:

- Skin type: Oily | Dry | Combination | Normal | Sensitive
- Primary condition flags: Acne | Hyperpigmentation | Uneven tone | Dehydration | None detected

3.3 Fine-Tuning Pipeline (Keras Code Architecture)

The training pipeline will follow a two-phase fine-tuning strategy, implemented in Python using Keras/TensorFlow:

```
Phase 1 – Feature Extraction:
```

- Freeze all EfficientNetB0 base layers (load with include_top=False, weights='imagenet').
- Attach a custom classification head: GlobalAveragePooling2D → BatchNormalization → Dense(256, relu) → Dropout(0.4) → Dense(n_classes, softmax).
- Train only the head for 15–20 epochs using Adam optimizer, categorical_crossentropy loss, and a learning rate of 1e-3.

```
Phase 2 – Fine-Tuning:
```

- Unfreeze the top 30 layers of EfficientNetB0.
- Continue training with a reduced learning rate of 1e-5 for 10–15 epochs.

- Apply data augmentation (RandomFlip, RandomRotation, RandomBrightness) to improve generalization across skin tones and lighting conditions.
- Use EarlyStopping and ReduceLROnPlateau callbacks to prevent overfitting.

3.4 Model Export & Deployment

After training, the model is exported in two formats for different deployment contexts:

Export Format	Use Case	Tool
TensorFlow Lite (.tflite)	On-device inference in React Native app	tf.lite.TFLiteConverter.from_keras_model()
TF SavedModel	Server-side inference via FastAPI backend	model.save('saved_model/')
ONNX (optional)	Cross-platform compatibility fallback	tf2onnx converter

Quantization: The TFLite model will be quantized (post-training dynamic range quantization) to reduce size from ~20MB to ~5MB, enabling faster inference and lower memory usage on mid-range Android devices common in the Kenyan market.

4. React Native Application Architecture

4.1 Technology Stack

Component	Technology	Purpose
Framework	React Native (Expo managed workflow)	Cross-platform iOS/Android app
AI Inference	react-native-tflite	Run .tflite model on-device
Camera	expo-camera / react-native-vision-camera	Capture skin images for analysis
State Management	Redux Toolkit + RTK Query	Global state & API data fetching
Navigation	React Navigation v6	Screen routing
Styling	NativeWind (Tailwind for RN)	Consistent UI styling
HTTP Client	Axios	FastAPI backend communication
Local Storage	AsyncStorage + SQLite	Offline profile caching
Auth	JWT + Secure Store	User authentication & session management
Push Notifications	Expo Notifications	Promotions, loyalty rewards alerts

4.2 Key Application Screens & Modules

The app is organized into five functional modules:

- Skin Analysis Module: Camera screen with guided overlay → image preprocessing (resize to 224×224, normalize) → on-device TFLite inference → display results (skin type, condition flags, confidence score).
- Questionnaire Module: Adaptive questionnaire (skin feel, routine, concerns) → combines with model output for higher-confidence final skin profile → stored to user account.
- Product Recommendation Module: Recommendation engine maps skin profile to Dr Rashel / Estelin product SKUs → displays ranked product cards with condition-to-ingredient explanations.
- Ordering Module: Retail and wholesale cart flows, M-Pesa and card payment integration, order tracking, synced with drrashel.co.ke inventory via backend API.
- Profile & Loyalty Module: Skin history tracking over time, progress photos, loyalty points dashboard, referral rewards, push notification opt-ins.

4.3 On-Device Inference Flow

The following sequence describes how a skin analysis is performed on-device:

1. User opens camera screen → expo-camera captures a still frame.
2. Image is pre-processed in JavaScript: resized to 224×224px, converted to Float32Array, normalized to [0, 1].

3. react-native-tflite runs the quantized .tflite model on the input tensor.
4. Output tensor (softmax probabilities) is decoded to skin type label + condition flags.
5. Result is combined with questionnaire answers and sent to FastAPI backend for recommendation generation.
6. Backend returns ranked product recommendations → displayed to user.

If the device cannot run on-device inference (low memory), the app falls back to sending the image to the FastAPI backend, which runs the SavedModel version of the Keras classifier server-side.

5. FastAPI Backend Architecture

5.1 Backend Services

Service / Endpoint	Method	Description
/api/v1/analyze	POST	Accepts image + questionnaire payload; runs SavedModel inference; returns skin profile JSON
/api/v1/recommend	POST	Accepts skin profile; queries recommendation engine; returns ranked product list
/api/v1/profile	GET/PUT	Retrieve and update user skin profile history
/api/v1/orders	POST/GET	Create and retrieve retail/wholesale orders; syncs with drrashel.co.ke
/api/v1/sync/bitmoji	POST	Ingest in-store Bitmoji analyzer scan result into user profile
/api/v1/loyalty	GET/POST	Loyalty points balance and transaction history
/api/v1/products	GET	Product catalog with stock levels from website inventory

5.2 Recommendation Engine

The recommendation engine is a rule-augmented content-based filtering system. It operates in two steps:

- Step 1 – Profile-to-ingredient mapping: A curated knowledge base maps each skin type and condition flag to beneficial active ingredients (e.g., Oily + Acne → Salicylic Acid, Niacinamide; Dry → Hyaluronic Acid, Ceramides).
- Step 2 – Product scoring: Each Dr Rashel / Estelin product SKU is tagged with its key ingredients. Products are scored based on ingredient overlap with the user's ideal ingredient set and ranked by match score.

In later iterations, this rule-based engine can be replaced or augmented with a collaborative filtering model (Keras Embedding layers) trained on anonymized purchase and satisfaction data.

5.3 Backend Infrastructure

Component	Technology
API Framework	FastAPI (Python 3.11)
ML Runtime	TensorFlow 2.x (serves Keras SavedModel)
Database	PostgreSQL (user profiles, orders, loyalty)
Cache	Redis (product catalog, recommendation cache)
Object Storage	AWS S3 / Cloudflare R2 (skin images, model artifacts)
Hosting	Docker + Railway or Render (affordable for academic project)
Authentication	JWT with refresh tokens

Component	Technology
Monitoring	Sentry (errors) + Prometheus + Grafana (metrics)

6. In-Store Bitmoji Analyzer Integration

The existing in-store Bitmoji skin analyzer at Dr Rashel Kenya's Nairobi store is a key asset for both training data collection and customer profile continuity.

6.1 Training Data Pipeline

With customer consent, in-store Bitmoji scan results (skin type classifications, condition reports) will be exported and used as labelled training data to fine-tune the Keras model. This is particularly valuable as it provides data representative of East African skin tones and conditions — an underrepresented segment in public datasets like ISIC.

- Data collection: Store staff export Bitmoji scan results as structured JSON/CSV.
- Label preparation: Bitmoji outputs are mapped to the standardized label schema (skin type + condition flags).
- Data pipeline: Labelled data is ingested into the Keras training pipeline via a preprocessing script that handles resizing, augmentation, and train/val/test splitting.

6.2 Profile Continuity

Customers who visit the store and have an in-store scan will have their profile synced to their mobile app account via the /api/v1/sync/bitmoji endpoint. This enables seamless continuity — a customer's in-store diagnosis history is visible in the app, and subsequent remote analyses are compared against their baseline.

7. Data Privacy & Security

Concern	Mitigation
Skin image storage	Images processed on-device where possible; if sent to server, deleted after inference within 24 hours
Personal data	Stored encrypted at rest (AES-256); transit encrypted via TLS 1.3
Consent	Explicit opt-in consent screen before any image capture or data sharing
GDPR / Kenya Data Protection Act 2019 compliance	Data minimization, right to deletion, privacy policy linked in app
Bitmoji training data	All in-store data anonymized before use in model training
Model security	.tflite model obfuscated; API endpoints rate-limited and authenticated

8. Testing & Validation Strategy

8.1 Model Validation

- Hold-out test set: 20% of dataset reserved for final model evaluation (accuracy, precision, recall, F1-score per class).
- Benchmark comparison: Mobile app analysis results compared against in-store Bitmoji analyzer outputs using Cohen's Kappa for agreement measurement.
- Bias audit: Model performance evaluated separately for different skin tone ranges (Fitzpatrick scale) to identify and correct demographic bias.

8.2 Application Testing

- Unit tests: Jest for React Native components and Redux slices.
- Integration tests: API endpoint testing with Pytest + httpx.
- End-to-end tests: Detox (React Native E2E framework) for core user flows.
- Usability testing: Pilot with 30 participants (15 in-store, 15 remote) collecting SUS (System Usability Scale) scores and qualitative feedback.

9. Implementation Roadmap

Phase	Duration	Key Deliverables
Phase 1: Data Collection & Preparation	Weeks 1–4	Bitmoji data export, ISIC dataset download, labelling pipeline, train/val/test split
Phase 2: Keras Model Training	Weeks 5–8	EfficientNetB0 fine-tuning (Phase 1 + Phase 2), model evaluation, TFLite export & quantization
Phase 3: Backend Development	Weeks 7–10	FastAPI setup, inference endpoint, recommendation engine, database schema, Dr Rashel API integration
Phase 4: React Native App	Weeks 9–14	All 5 app modules, TFLite on-device inference integration, UI/UX, M-Pesa payment
Phase 5: Integration & Testing	Weeks 13–16	End-to-end integration, model benchmark vs Bitmoji, usability pilot, bug fixing
Phase 6: Dissertation & Submission	Weeks 17–20	Write-up: design, implementation, results, recommendations, academic contribution

10. Risks & Mitigation

Risk	Likelihood	Impact	Mitigation
Insufficient labelled skin data	Medium	High	Supplement with ISIC dataset + synthetic augmentation; lower class count if needed
TFLite model too large for low-end devices	Medium	Medium	Apply INT8 quantization; fallback to server-side inference

Risk	Likelihood	Impact	Mitigation
react-native-tflite compatibility issues	Low	High	Pin dependency versions; test on both iOS and Android early in Phase 4
Bitmoji data export not available	Low	Medium	Rely on ISIC + questionnaire labels; collect Bitmoji data post-launch
Dr Rashel API integration delays	Medium	Medium	Mock API for development; integrate website in Phase 5
Model bias across skin tones	Medium	High	Stratified sampling by Fitzpatrick scale; bias audit in validation phase

11. Conclusion

This architecture establishes a robust, scalable, and academically rigorous technical foundation for the AI skin analysis mobile application. By integrating a Keras fine-tuned EfficientNetB0 classifier with a React Native front-end and a FastAPI backend, the system avoids the prohibitive cost and risk of training a vision model from scratch, while still delivering a model tailored to the specific skin conditions and demographics served by Dr Rashel Kenya.

The use of transfer learning, on-device TFLite inference, and a structured recommendation engine makes the solution both technically sophisticated and practically deployable within an MSc project scope. The Bitmoji integration provides a unique and commercially valuable feedback loop between in-store expertise and remote AI-driven services — a contribution that is both technically and academically meaningful.

This document will be updated iteratively as implementation progresses through each phase.